GUIA TÉCNICO

Atenção, Transformers e tendências em Deep Learning

Capítulo 20/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Derivar atenção escalada e entender as diferenças entre arquiteturas recorrentes e Transformers.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Atenção mede relações entre tokens por similaridades de consultas e chaves, normalizadas por softmax, e combina valores. Atenção causal bloqueia posições futuras. Transformers acrescentam positional encoding, normalização, resíduos e feed-forward. Aprofundamento aplicado de PLN está na área de Processamento de Linguagem Natural.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: $\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d_k})V$

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Atenção com Q/K/V, máscara causal e softmax estável é o componente; Transformer inclui blocos residuais e posição.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
q=np.array([[1.,0.],[0.,1.]]);k=q.copy();v=np.array([[2.,1.],[0.,3.]])
scores=q@k.T/np.sqrt(q.shape[-1]);p=np.exp(scores-scores.max(axis=-1,keepdims=True));p/=p.  ↪
    sum(axis=-1,keepdims=True)
out=p@v
print('Pesos de atenção:\n',np.round(p,3),'\nSaída:',np.round(out,3))
assert np.allclose(p.sum(axis=-1),1)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados

reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Atenção escalada com máscara causal

Atenção com Q/K/V, máscara causal e softmax estável é o componente; Transformer inclui blocos residuais e posição.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
X=np.array([[1.,0.,0.],[0.,1.,0.],[0.,0.,1.],[1.,1.,0.]])
q=k=v=X
logits=q@k.T/np.sqrt(X.shape[1]);causal=np.triu(np.ones_like(logits,dtype=bool),k=1)
logits=np.where(causal,-1e9,logits)
weights=np.exp(logits-logits.max(axis=1,keepdims=True));weights/=weights.sum(axis=1,keepdi ↪
    ms=True)
out=weights@v
assert out.shape==X.shape and np.allclose(weights.sum(axis=1),1)
assert np.allclose(weights[causal],0)
print('Pesos causais (arredondados):\n',weights.round(3))
```

A máscara impede o acesso a tokens futuros durante decodificação autorregressiva; softmax precisa ser numericamente estável e o alinhamento de sequências deve ser correto.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que dividir o produto QK^T por $\sqrt{d_k}$?

Resposta: Para controlar a escala dos logits, reduzindo saturação da softmax quando a dimensão cresce.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide

REVISÃO DIDÁTICA · CAPÍTULO 20

Scaled dot-product attention e Transformer

1. Conceito, escopo e interpretação

Atenção escalada pondera valores V pela similaridade entre consultas Q e chaves K. Uma máscara causal impede que a posição t atenda a posições futuras. O mecanismo de atenção é parte de um Transformer, mas não equivale sozinho à arquitetura completa.

2. Formalização e exemplo resolvido

> Attention(Q,K,V)=softmax(Q K^T/sqrt(d_k)+M)V. Na máscara causal, M_ij=-infinito se j>i, e 0 caso contrário. Cada linha de softmax soma aproximadamente 1 sobre posições permitidas.

3. Relação com o notebook e experimento

No notebook reproduza Q/K/V em NumPy e verifique manualmente pesos, normalização e efeito da máscara causal em uma sequência curta. Um Transformer inclui também embeddings posicionais, blocos residuais, normalização e feed-forward.

4. Limitações e erros frequentes

Não chame a simulação de atenção de implementação de Transformer completo. Padding e máscara causal resolvem problemas diferentes e podem ser combinados conforme a tarefa.

5. Exercício comentado

Por que a primeira posição de uma sequência autoregressiva só pode atender a si mesma? Para não receber informação de tokens futuros durante a previsão.

6. Ampliação conceitual do guia

A atenção usa softmax(Q*K^T/sqrt(d_k))V para ponderar valores de acordo com a compatibilidade consulta-chave. A máscara causal impede atenção a posições futuras em modelos autoregressivos; ela deve ser aplicada aos scores antes do softmax.

7. Orientação de estudo e validação

O notebook implementa pesos de atenção com NumPy e máscara causal. Isso não é um Transformer completo: faltam camadas de projeção treináveis, multi-head, conexões residuais, normalização, feed-forward e codificação posicional. Diferencie a operação isolada da arquitetura.

8. Exercício adicional de fixação

Exercício: por que dividir pelo sqrt(d_k)? Resposta: reduz a escala dos produtos escalares e ajuda a evitar saturação excessiva do softmax.

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório

LABORATÓRIO COMPLEMENTAR · CAPÍTULO 20

Atenção escalada com máscara causal — NumPy

Implementação ilustrativa / formalização

```
import numpy as np
Q=np.array([[1.,0.],[0.,1.],[1.,1.]])
K=Q.copy(); V=np.arange(6.).reshape(3,2)
S=(Q@K.T)/np.sqrt(Q.shape[-1])
mask=np.triu(np.ones((3,3),dtype=bool),k=1)
S=np.where(mask,-np.inf,S)
exp=np.exp(S-np.max(S,axis=-1,keepdims=True))
A=exp/exp.sum(axis=-1,keepdims=True)
O=A@V
assert np.allclose(A.sum(axis=1),1)
assert np.allclose(np.triu(A,k=1),0)
print('Pesos causais:',A,'Saidas:',O,sep='\n')
```

Interpretação e limites

Este laboratório calcula atenção causal single-head em NumPy; um Transformer completo precisa de projeções treináveis, representação posicional, blocos feed-forward, normalização e conexões residuais. Teste também máscaras de padding quando houver sequências de comprimentos diferentes.

Como verificar

Execute o código no ambiente indicado pelo notebook. Se depender de Keras/TensorFlow, registre versões e resultados reais; esta revisão não afirma execução do laboratório.

Referência: notebook do mesmo capítulo em 07_deep_learning (GitHub).